Arquitectura · Tareas · Prompts · Código

Versión 1.0 · Marzo 2026 · Documento técnico de referencia

1. Arquitectura del sistema

El sistema transforma inputs de marca (logo, URL, brand book, texto) en 6 mockups fotorrealistas mediante 4 capas en pipeline. Ninguna capa usa imagen generativa. Todo el resultado es determinista excepto los design briefs JSON, que emergen del razonamiento de Claude.

Capa	Qué hace	Tecnología	Input	Output
0 — Ingesta	Recoge y normaliza todos los inputs del cliente	Python + web_fetch	Logo, URL, brand book PDF, texto	Assets normalizados
1 — Agente IA	Infiere vocabulario visual y genera 6 design briefs JSON	Claude Sonnet 4.5 API	Assets normalizados	6× JSON de diseño
2 — Renderer	Dibuja el diseño plano desde el JSON (fondo + recursos + texto)	Python · Pillow · svgpathtools	JSON + assets	6× imagen plana
3 — Compositing	Proyecta el diseño plano sobre la foto del trofeo	Python · OpenCV · NumPy	Imagen plana + foto trofeo	6× mockup final
4 — QA	Valida calidad y dispara corrección si falla	Python · PIL rules	Imagen plana + mockup	Pass/fail + refeed

→ FLUJO COMPLETO: Input → Capa 0 → Capa 1 (×6 en paralelo) → Capa 2 (×6) → Capa 3 (×6) → Capa 4 (×6). Si Capa 4 falla: refeed a Capa 1 con error context. Máximo 3 iteraciones por propuesta. Tiempo total estimado: 30–90 segundos para 6 mockups.

Estructura de datos entre capas

```
# CAPA 0 → CAPA 1
normalized_input = {
    "logo_svg": "<svg>...</svg>",          # SVG raw o None si es PNG
    "logo_png_b64": "base64...",            # siempre disponible
    "logo_colors": ["#1B3A4B", "#FFFFFF"],  # extraídos por Capa 0
    "logo_shapes": ["circle", "arc"],       # geometría detectada
    "web_content": "...",                   # texto + estructura de la URL
    "web_colors": ["#HEX", ...],            # colores dominantes de la web
    "brandbook_text": "...",                # texto extraído del PDF
    "brandbook_images_b64": ["..."],        # páginas como imágenes
    "award_text": {
        "event": "SKO FY27",
        "category": "Account Executive of the Year",
        "recipient": "Ryan Leventhal",
```

```
        "date": "FY26",
        "edition": "Field First"
    },
    "trophy_model": "totem_basic",
    "trophy_constraints": {
        "print_area_mm": [40, 120],
        "orientation": "portrait",
        "safe_zone_mm": 3
    }
}

# CAPA 1 → CAPA 2  (×6)
design_brief = {
    "proposal_id": "p1",
    "background_type": "solid",
    "background_color_1": "#1C2B3A",
    "illustration_1": { "active": True, "style": "figurative_hero", ... },
    "illustration_2": { "active": False, ... },
    "brand_shape":    { "active": True, "shape_type": "circle", ... },
    "pattern_bg":     { "active": False, ... },
    "split_layout":   { "active": False, ... },
    "color_band":     { "active": True, "position": "bottom", ... },
    "decorative_lines":{ "active": False, ... },
    "typography_as_graphic": { "active": False, ... },
    "text_blocks": [...],
    "logo_treatment": "reversed",
    "design_rationale": "..."
}
```

CAPA 0 — INGESTA Y NORMALIZACIÓN DE INPUTS

Tareas

— **T0.1** Extracción de colores del logo SVG/PNG
— **T0.2** Análisis de geometría del logo
— **T0.3** Fetch y análisis de URL corporativa
— **T0.4** Extracción de texto e imágenes del brand book PDF
— **T0.5** Ensamblado del normalized_input

T0.1 + T0.2 — Análisis de logo

```
# requirements: pillow, svgpathtools, colorthief, lxml, cairosvg

import re, base64, cairosvg
from PIL import Image
from io import BytesIO
from colorthief import ColorThief
from lxml import etree

def analyze_logo(svg_content: str | None, png_bytes: bytes) -> dict:
    colors = extract_colors_png(png_bytes)
    shapes = []

    if svg_content:
        shapes = extract_svg_shapes(svg_content)
        # render SVG to PNG for color extraction if no PNG
        if not png_bytes:
            png_bytes = cairosvg.svg2png(bytestring=svg_content.encode())

    return {
        "logo_svg": svg_content,
        "logo_png_b64": base64.b64encode(png_bytes).decode(),
        "logo_colors": colors,
        "logo_shapes": shapes,
    }

def extract_colors_png(png_bytes: bytes) -> list[str]:
    img = Image.open(BytesIO(png_bytes)).convert("RGBA")
    # remove transparent pixels before color extraction
    bg = Image.new("RGB", img.size, (255, 255, 255))
    bg.paste(img, mask=img.split()[3])
    ct = ColorThief(BytesIO(png_bytes))
    palette = ct.get_palette(color_count=5, quality=1)
    return [f"#{r:02X}{g:02X}{b:02X}" for r,g,b in palette]

def extract_svg_shapes(svg_content: str) -> list[str]:
    """Detect dominant geometric primitives in SVG."""
    tree = etree.fromstring(svg_content.encode())
    ns = {"svg": "http://www.w3.org/2000/svg"}
    shapes = set()
    tag_map = {
        "circle": "circle", "ellipse": "circle",
        "rect":   "rectangle", "polygon": "polygon",
        "path":   "path",  "line": "line",
    }
    for el in tree.iter():
        tag = el.tag.split("}")[-1]
        if tag in tag_map:
            shapes.add(tag_map[tag])
    # heuristic: detect arc/semicircle from path d attribute
```

```
    for path in tree.iter("{*}path"):
        d = path.get("d", "")
        if d.count("A") >= 2 or d.count("a") >= 2:
            shapes.add("arc")
    return list(shapes)
```

T0.3 — Fetch de URL corporativa

```
import httpx
from bs4 import BeautifulSoup
from colorthief import ColorThief

async def analyze_corporate_url(url: str) -> dict:
    async with httpx.AsyncClient(follow_redirects=True) as client:
        r = await client.get(url, timeout=15)
    soup = BeautifulSoup(r.text, "html.parser")

    # extract text content (nav, hero, about sections)
    text = " ".join([
        el.get_text(strip=True)
        for el in soup.select("h1,h2,h3,p,nav,header")
    ])[:4000]  # limit tokens

    # extract CSS color values
    style_tags = soup.find_all("style")
    hex_pattern = re.compile(r"#(?:[0-9a-fA-F]{3}){1,2}\b")
    css_colors = list(set(hex_pattern.findall(",".join(t.text for t in style_tags))))

    # detect visual density from element count
    img_count = len(soup.find_all("img"))
    visual_density = "rich" if img_count > 10 else "minimal"

    return {
        "web_content": text,
        "web_colors": css_colors[:10],
        "visual_density": visual_density,
    }
```

T0.4 — Extracción de brand book PDF

```
import fitz  # PyMuPDF
import base64

def extract_brandbook(pdf_bytes: bytes) -> dict:
    doc = fitz.open(stream=pdf_bytes, filetype="pdf")
    text_pages = []
    image_pages = []

    for i, page in enumerate(doc):
        # text extraction
        text_pages.append(page.get_text())

        # render pages as images for vision (max 6 pages)
        if i < 6:
            pix = page.get_pixmap(dpi=120)
            img_b64 = base64.b64encode(pix.tobytes("png")).decode()
            image_pages.append(img_b64)

    return {
        "brandbook_text": " ".join(text_pages)[:6000],
        "brandbook_images_b64": image_pages,
```

```
    }
```

CAPA 1 — AGENTE DE DISEÑO · Prompts y orquestación

La Capa 1 tiene 3 llamadas a Claude en secuencia. Separar el análisis de la generación mejora la calidad: Claude analiza primero con profundidad y luego genera con contexto completo.

Llamada	Objetivo	Input	Output
1A — Análisis de marca	Extraer vocabulario visual de logo + URL + brand book	Assets normalizados + imágenes	brand_analysis JSON
1B — Selección de arquetipos	Elegir 6 combinaciones de recursos gráficos distintas	brand_analysis + trophy constraints	archetyp_plan JSON (6 slots)
1C — Design briefs	Generar el JSON completo por cada propuesta	archetype_plan + brand_analysis	6× design_brief JSON

LLAMADA 1A — Análisis de marca

System prompt

```
You are a senior brand analyst specializing in visual identity systems.
Your role is to analyze a brand's visual assets and extract a precise,
structured description of its graphic vocabulary.

You must output ONLY valid JSON. No preamble, no markdown, no explanation.

Analyze with extreme attention to:
- The geometric primitives in the logo (circles, arcs, lines, organic shapes)
- Which logo elements could work independently as decorative assets
- The color hierarchy: primary action color, secondary, neutral, background
- Whether the brand uses typography as a graphic element (bold, display, rotated)
- Visual density of the brand (minimal / moderate / rich)
- Whether gradients, patterns, or geometric shapes are part of the brand system
- Tone: corporate / playful / technical / luxury / activist

Be specific about colors (exact HEX). Never approximate.
If you detect a color from an image, extract it precisely.
```

User prompt — Llamada 1A

```
Analyze the following brand assets and return a brand_analysis JSON object.

LOGO COLORS DETECTED: {{ logo_colors }}
LOGO GEOMETRIC SHAPES DETECTED: {{ logo_shapes }}

CORPORATE WEBSITE TEXT ({{ url }}):
{{ web_content }}

WEBSITE CSS COLORS: {{ web_colors }}

BRAND BOOK TEXT:
{{ brandbook_text }}

Return this exact structure:
```

```
{
  "brand_name": "...",
  "brand_tone": "corporate|playful|technical|luxury|activist",
  "visual_density": "minimal|moderate|rich",
  "colors": {
    "primary": "#HEX",
    "secondary": "#HEX|null",
    "accent": "#HEX|null",
    "background_preferred": "#HEX",
    "text_on_dark": "#HEX",
    "text_on_light": "#HEX"
  },
  "typography": {
    "primary_font": "font name or null",
    "style": "serif|sans-serif|display|monospace",
    "weight_preference": "light|regular|bold|black",
    "uses_typography_as_graphic": true|false
  },
  "logo_analysis": {
    "type": "symbolic|typographic|combined",
    "has_extractable_symbol": true|false,
    "symbol_description": "...",
    "dominant_shape": "circle|rectangle|triangle|organic|line|arc|none",
    "can_derive_decorative_shape": true|false,
    "derived_shape": "circle|semicircle|arc|triangle|fragment|none"
  },
  "graphic_resources": {
    "uses_gradients": true|false,
    "uses_patterns": true|false,
    "uses_geometric_bg": true|false,
    "uses_illustration": true|false,
    "uses_bold_color_bg": true|false
  },
  "layout_preferences": {
    "text_alignment": "left|center|right",
    "uses_split_layout": true|false,
    "uses_color_bands": true|false
  }
}
```

Código de llamada 1A

```
import anthropic, base64, json

client = anthropic.Anthropic()  # API key from env

def call_1a_brand_analysis(normalized_input: dict) -> dict:
    # Build vision content blocks from brandbook images
    content = []

    # Add logo image
    content.append({
        "type": "image",
        "source": {
            "type": "base64",
            "media_type": "image/png",
            "data": normalized_input["logo_png_b64"]
        }
    })

    # Add up to 3 brandbook pages as images
```

```
    for img_b64 in normalized_input.get("brandbook_images_b64", [])[:3]:
        content.append({
            "type": "image",
            "source": {"type": "base64", "media_type": "image/png", "data": img_b64}
        })

    # Add text prompt
    user_text = USER_PROMPT_1A.format(
        logo_colors=normalized_input["logo_colors"],
        logo_shapes=normalized_input["logo_shapes"],
        url=normalized_input.get("source_url", ""),
        web_content=normalized_input["web_content"],
        web_colors=normalized_input["web_colors"],
        brandbook_text=normalized_input["brandbook_text"],
    )
    content.append({"type": "text", "text": user_text})

    response = client.messages.create(
        model="claude-sonnet-4-5-20251101",
        max_tokens=1500,
        system=SYSTEM_PROMPT_1A,
        messages=[{"role": "user", "content": content}]
    )

    raw = response.content[0].text
    return json.loads(raw)  # validated downstream with Pydantic
```

LLAMADA 1B — Selección de arquetipos

Decide qué 6 combinaciones de recursos gráficos usar. La regla clave: ninguna propuesta puede repetir el mismo structural_background + mismo patrón dominante.

System prompt

> You are a creative director selecting graphic design directions for an award trophy.
> Given a brand analysis, you will select 6 distinct compositional approaches.
>
> RULES — strictly enforced:
> 1. Each proposal must use a different structural_background (solid/gradient/split/white)
> 2. Each proposal must have a different dominant graphic resource
> 3. Always include: 1 minimal proposal (no illustration), 1 bold-color proposal
> 4. At least 2 proposals with light backgrounds and 2 with dark backgrounds
> 5. Never activate illustration_1 and typography_as_graphic simultaneously
> 6. pattern_bg and illustration_1 can coexist if illustration_1 is in foreground
> 7. Output ONLY valid JSON. No explanation.

User prompt — Llamada 1B

> BRAND ANALYSIS:
> {{ brand_analysis_json }}
>
> TROPHY MODEL: {{ trophy_model }}
> TROPHY ORIENTATION: {{ orientation }} (portrait|landscape)
>
> Select 6 archetype combinations. For each, specify which graphic resources
> are active and what their primary parameters should be.
>
> Return an array of 6 objects:

```
[
  {
    "proposal_id": "p1",
    "label": "Bold & Chromatic",
    "structural_background": "solid|gradient|split_h|split_v|white",
    "dominant_resource": "illustration_hero|typography_graphic|pattern_bg|brand_shape|
color_band|minimal",
    "active_resources": ["illustration_1", "color_band"],
    "background_dark": true,
    "design_rationale": "one sentence in Spanish explaining the creative direction"
  },
  ...
]
```

LLAMADA 1C — Design briefs completos

Genera el JSON completo para cada propuesta. Se hace en 1 llamada pidiendo los 6 simultáneamente para aprovechar el contexto cacheado del brand_analysis.

System prompt

```
You are a senior graphic designer creating precise design specifications for trophy awards.
You receive a brand analysis and an archetype plan, and you generate
exact, complete JSON design briefs that a rendering engine will execute directly.

CRITICAL: Every HEX color must come from the brand analysis. Never invent colors.
Every font name must be exact or use the fallback "system_sans".
Coordinates and scales are always relative to the print area (0.0 to 1.0).
opacity values are always 0.0 to 1.0.
Output ONLY the JSON array of 6 design_brief objects. No explanation.
```

User prompt — Llamada 1C

```
BRAND ANALYSIS: {{ brand_analysis_json }}
ARCHETYPE PLAN: {{ archetype_plan_json }}
AWARD TEXT: {{ award_text_json }}
TROPHY CONSTRAINTS: {{ trophy_constraints_json }}

Generate the complete design_brief for each of the 6 proposals.
Follow each proposal's active_resources list strictly.

For each proposal return:
{
  "proposal_id": "p1",
  "background_type": "solid|gradient|split_h|split_v|white",
  "background_color_1": "#HEX",
  "background_color_2": "#HEX|null",
  "gradient_direction": "top_bottom|diagonal_tl|null",
  "illustration_1": { "active": bool, "style": "...", "placement": "...", "scale": float, "opacity": float,
"color_mode": "..." },
  "illustration_2": { "active": bool, ... },
  "brand_shape": { "active": bool, "shape_type": "...", "placement": "...", "scale": float, "color":
"#HEX", "count": int },
  "pattern_bg": { "active": bool, "type": "...", "coverage": "...", "opacity": float, "element_count":
int },
  "split_layout": { "active": bool, "direction": "...", "ratio": "...", "color_zone_1": "#HEX",
```

```
"color_zone_2": "#HEX", "content_zone_1": "...", "content_zone_2": "..." },
  "color_band": { "active": bool, "position": "top|bottom", "height_percent": int, "color": "#HEX",
"content": "logo|none" },
  "decorative_lines": { "active": bool, "style": "...", "placement": "...", "color": "#HEX" },
  "typography_as_graphic": { "active": bool, "text_source": "...", "rotation": int, "size_mode": "...",
"placement": "...", "color": "#HEX" },
  "text_color": "#HEX",
  "text_alignment": "left|center|right",
  "logo_treatment": "standard|reversed|monochrome",
  "logo_position": "top_left|top_center|bottom_left",
  "logo_scale": float,
  "headline_weight": "light|regular|bold|black",
  "headline_size_ratio": float,
  "secondary_rule": { "active": bool },
  "margin_breathing": "compact|standard|generous",
  "design_rationale": "..."
}
```

Código de orquestación Capa 1

```
async def run_layer_1(normalized_input: dict) -> list[dict]:
    # Llamada 1A — brand analysis
    brand_analysis = call_1a_brand_analysis(normalized_input)
    validate_brand_analysis(brand_analysis)  # Pydantic

    # Llamada 1B — archetype selection
    archetype_plan = call_1b_archetype_plan(
        brand_analysis,
        trophy_model=normalized_input["trophy_model"],
        constraints=normalized_input["trophy_constraints"],
    )

    # Llamada 1C — design briefs (6 en una sola llamada)
    briefs = call_1c_design_briefs(
        brand_analysis,
        archetype_plan,
        award_text=normalized_input["award_text"],
        constraints=normalized_input["trophy_constraints"],
    )

    # Validate all 6 briefs
    for brief in briefs:
        validate_design_brief(brief)  # Pydantic model

    return briefs  # list of 6 dicts
```

CAPA 2 — RENDERER · Dibujado programático del diseño plano

Cada función de renderizado recibe el design_brief JSON y dibuja su capa sobre un canvas Pillow. El orden de composición es fijo: fondo → recursos gráficos → logo → texto.

```
# Canvas size: siempre en píxeles equivalentes a 300dpi del área imprimible
# Totem Basic print area: 40×120mm @ 300dpi = 472×1417px
# Placa A4 print area: 190×260mm @ 300dpi = 2244×3071px
# Ratio siempre se mantiene. Todo coordinate es relativo (0.0-1.0).

from PIL import Image, ImageDraw, ImageFont, ImageFilter, ImageChops
import numpy as np

def render_design(brief: dict, assets: dict, canvas_px: tuple) -> Image:
    W, H = canvas_px
    canvas = Image.new("RGBA", (W, H), (255,255,255,255))

    canvas = render_background(canvas, brief)
    canvas = render_pattern_bg(canvas, brief, assets)
    canvas = render_illustration_2(canvas, brief, assets)  # bajo
    canvas = render_split_layout(canvas, brief)
    canvas = render_color_band(canvas, brief)
    canvas = render_typography_as_graphic(canvas, brief)
    canvas = render_illustration_1(canvas, brief, assets)  # hero
    canvas = render_decorative_lines(canvas, brief)
    canvas = render_brand_shape(canvas, brief)
    canvas = render_logo(canvas, brief, assets)
    canvas = render_text_blocks(canvas, brief)
    canvas = render_secondary_rule(canvas, brief)

    return canvas
```

Render: fondo (solid / gradient / white)

```
def render_background(canvas: Image, brief: dict) -> Image:
    W, H = canvas.size
    draw = ImageDraw.Draw(canvas)
    bg_type = brief["background_type"]
    c1 = hex_to_rgb(brief["background_color_1"])

    if bg_type == "white" or bg_type == "solid":
        canvas.paste(Image.new("RGBA", (W,H), (*c1, 255)))

    elif bg_type == "gradient":
        c2 = hex_to_rgb(brief.get("background_color_2", "#FFFFFF"))
        direction = brief.get("gradient_direction", "top_bottom")
        arr = np.zeros((H, W, 4), dtype=np.uint8)

        for i in range(H if direction == "top_bottom" else W):
            t = i / (H if direction == "top_bottom" else W)
            r = int(c1[0] + t * (c2[0] - c1[0]))
            g = int(c1[1] + t * (c2[1] - c1[1]))
            b = int(c1[2] + t * (c2[2] - c1[2]))
            a = int(255 + t * (int(brief.get("gradient_end_opacity",0)*255) - 255))
            if direction == "top_bottom":
                arr[i, :] = [r, g, b, 255]
            else:
                arr[:, i] = [r, g, b, 255]

        # diagonal variants
        if "diagonal" in direction:
```

```
            from PIL import ImageOps
            arr_img = Image.fromarray(arr)
            arr_img = arr_img.rotate(-45, expand=True).crop((0,0,W,H))
            canvas.paste(arr_img)
        else:
            canvas.paste(Image.fromarray(arr))

    return canvas
```

Render: pattern_bg (radial / scatter / sunburst)

```
import math

def render_pattern_bg(canvas: Image, brief: dict, assets: dict) -> Image:
    cfg = brief.get("pattern_bg", {})
    if not cfg.get("active"): return canvas

    W, H = canvas.size
    layer = Image.new("RGBA", (W, H), (0,0,0,0))
    draw  = ImageDraw.Draw(layer)
    color = hex_to_rgb(cfg.get("color", "#000000"))
    alpha = int(cfg.get("opacity", 0.15) * 255)
    count = cfg.get("element_count", 16)
    ptype = cfg.get("type", "scatter_float")

    if ptype == "sunburst" or ptype == "radial":
        cx, cy = W // 2, H // 2
        for i in range(count):
            angle = (360 / count) * i
            rad   = math.radians(angle)
            x1 = cx + 20 * math.cos(rad)
            y1 = cy + 20 * math.sin(rad)
            x2 = cx + (min(W,H) * 0.55) * math.cos(rad)
            y2 = cy + (min(W,H) * 0.55) * math.sin(rad)
            draw.line([(x1,y1),(x2,y2)], fill=(*color, alpha), width=max(2, W//120))

    elif ptype == "scatter_float":
        import random; random.seed(42)
        for _ in range(count):
            r  = random.randint(W//20, W//5)
            x  = random.randint(-r, W+r)
            y  = random.randint(-r, H+r)
            draw.ellipse([x-r, y-r, x+r, y+r], outline=(*color, alpha),
width=max(1,W//200))

    elif ptype == "concentric":
        cx, cy = W//2, H//2
        for i in range(1, count+1):
            r = (min(W,H) // (count+1)) * i
            draw.ellipse([cx-r, cy-r, cx+r, cy+r], outline=(*color, alpha),
width=max(1,W//200))

    canvas = Image.alpha_composite(canvas, layer)
    return canvas
```

Render: color_band

```
def render_color_band(canvas: Image, brief: dict) -> Image:
    cfg = brief.get("color_band", {})
    if not cfg.get("active"): return canvas

    W, H  = canvas.size
```

```
    color = hex_to_rgb(cfg["color"])
    h_pct = cfg.get("height_percent", 22) / 100
    bh    = int(H * h_pct)
    pos   = cfg.get("position", "bottom")

    band = Image.new("RGBA", (W, bh), (*color, 255))
    y    = H - bh if pos == "bottom" else 0
    canvas.paste(band, (0, y))
    return canvas
```

Render: tipografía rotada como gráfico

```
def render_typography_as_graphic(canvas: Image, brief: dict) -> Image:
    cfg = brief.get("typography_as_graphic", {})
    if not cfg.get("active"): return canvas

    W, H   = canvas.size
    text   = brief["_award_text"][cfg.get("text_source","brand_name")]
    color  = hex_to_rgb(cfg.get("color","#FFFFFF"))
    rotation = cfg.get("rotation", 90)

    # compute font size to fill height
    font_size = H // len(text) * 2 if cfg.get("size_mode") == "fill_height" else H // 4
    font = load_font(brief.get("headline_font"), font_size)

    # render text on temp canvas then rotate
    tmp = Image.new("RGBA", (W*3, H*3), (0,0,0,0))
    td  = ImageDraw.Draw(tmp)
    td.text((W, H), text, font=font, fill=(*color, 180))
    tmp = tmp.rotate(rotation, expand=False)

    # crop to canvas size and composite
    offset_x = 0 if cfg.get("placement","left_column") == "left_column" else W//2
    region = tmp.crop((W - offset_x, H//2, W*2 - offset_x, H*3//2))
    region = region.resize((W, H))
    canvas = Image.alpha_composite(canvas, region)
    return canvas
```

Render: texto jerárquico

```
def render_text_blocks(canvas: Image, brief: dict) -> Image:
    W, H = canvas.size
    draw = ImageDraw.Draw(canvas)
    award = brief["_award_text"]
    margin_map = {"compact": 0.06, "standard": 0.09, "generous": 0.13}
    margin = int(W * margin_map.get(brief.get("margin_breathing","standard"), 0.09))
    align  = brief.get("text_alignment", "left")
    color  = hex_to_rgb(brief.get("text_color","#1A1A1A"))

    # base unit = W / 10
    base = W // 10
    ratio = brief.get("headline_size_ratio", 1.6)
    weight = brief.get("headline_weight", "bold")

    blocks = [
        {"role": "event",     "text": award.get("event",""),     "size": base, "w":
"regular"},
        {"role": "category",  "text": award.get("category",""),  "size": int(base*0.85),
"w": "light"},
        {"role": "recipient", "text": award.get("recipient",""), "size": int(base*ratio),
"w": weight},
        {"role": "date",      "text": award.get("date",""),      "size": int(base*0.7),
```

```
"w": "light"},
    ]

    # find vertical start — below logo zone
    logo_zone_end = int(H * 0.55)  # approx, refined after logo render
    y = logo_zone_end + margin

    for block in blocks:
        if not block["text"]: continue
        font = load_font(brief.get("headline_font"), block["size"])
        bbox = draw.textbbox((0,0), block["text"], font=font)
        tw   = bbox[2] - bbox[0]
        x    = margin if align == "left" else (W - tw)//2 if align == "center" else W -
tw - margin
        draw.text((x, y), block["text"], font=font, fill=(*color, 255))
        y   += block["size"] + int(block["size"] * 0.35)

    return canvas
```

CAPA 3 — COMPOSITING · Proyección sobre foto del trofeo

T3.1 — Calibración de homografía (trabajo manual por modelo)

Una sola vez por modelo de trofeo. Se ejecuta con una herramienta interactiva y el resultado se persiste en JSON.

```
# herramienta de calibración — ejecutar localmente, no en producción
import cv2, json, numpy as np

def calibrate_trophy(photo_path: str, model_id: str):
    """Click 4 corners of the printable surface en orden:
    top-left → top-right → bottom-right → bottom-left"""
    img  = cv2.imread(photo_path)
    pts  = []

    def click(event, x, y, flags, param):
        if event == cv2.EVENT_LBUTTONDOWN and len(pts) < 4:
            pts.append([x, y])
            cv2.circle(img, (x,y), 8, (0,255,0), -1)
            cv2.imshow("Calibrate", img)

    cv2.imshow("Calibrate", img)
    cv2.setMouseCallback("Calibrate", click)
    cv2.waitKey(0)

    if len(pts) == 4:
        homography_data = {
            "model_id": model_id,
            "photo_path": photo_path,
            "photo_size": [img.shape[1], img.shape[0]],
            "dst_points": pts  # 4 corners en la foto
        }
        with open(f"calibrations/{model_id}.json", "w") as f:
            json.dump(homography_data, f, indent=2)
        print(f"Calibration saved for {model_id}")
    cv2.destroyAllWindows()
```

T3.2 — Compositing

```
import cv2, numpy as np, json
from PIL import Image

def composite_mockup(
    design_img: Image,       # output de Capa 2, RGBA
    trophy_photo_path: str,
    model_id: str
) -> Image:

    # load calibration
    with open(f"calibrations/{model_id}.json") as f:
        cal = json.load(f)

    trophy = cv2.imread(trophy_photo_path)  # BGR
    H_photo, W_photo = trophy.shape[:2]

    # source points: corners del diseño plano
    W_d, H_d = design_img.size
    src_pts = np.float32([[0,0],[W_d,0],[W_d,H_d],[0,H_d]])
```

```
    dst_pts = np.float32(cal["dst_points"])

    # compute homography
    M, _ = cv2.findHomography(src_pts, dst_pts)

    # warp design onto photo canvas
    design_cv = cv2.cvtColor(np.array(design_img.convert("RGBA")), cv2.COLOR_RGBA2BGRA)
    warped    = cv2.warpPerspective(design_cv, M, (W_photo, H_photo))

    # split alpha and composite
    warped_bgr   = warped[:,:,:3]
    warped_alpha = warped[:,:,3:4] / 255.0

    # blend modes: multiply to integrate texture
    trophy_f = trophy.astype(np.float32) / 255.0
    warped_f = warped_bgr.astype(np.float32) / 255.0

    # multiply blend where design is present
    blended  = (trophy_f * (1.0 - warped_alpha * 0.15) +
                warped_f * warped_alpha * 0.85)
    blended  = np.clip(blended * 255, 0, 255).astype(np.uint8)

    # soft edge on design borders
    mask = Image.fromarray((warped_alpha[:,:,0] * 255).astype(np.uint8))
    mask = mask.filter(ImageFilter.GaussianBlur(radius=1.5))

    result_cv = blended
    result    = Image.fromarray(cv2.cvtColor(result_cv, cv2.COLOR_BGR2RGB))
    return result
```

CAPA 4 — VALIDACIÓN AUTÓNOMA DE CALIDAD

```
from PIL import ImageStat
from dataclasses import dataclass

@dataclass
class QAResult:
    passed: bool
    errors: list[str]
    warnings: list[str]

def run_qa(design_flat: Image, mockup: Image, brief: dict) -> QAResult:
    errors, warnings = [], []

    # 1. Contrast ratio — texto sobre fondo
    cr = check_contrast_ratio(design_flat, brief)
    if cr < 4.5:
        errors.append(f"CONTRAST_FAIL: ratio={cr:.2f} < 4.5 (WCAG AA).
text_color={brief['text_color']}")
    elif cr < 7.0:
        warnings.append(f"CONTRAST_LOW: ratio={cr:.2f}. Consider stronger contrast.")

    # 2. Text overflow — no text outside safe zone
    if check_text_overflow(design_flat, brief):
        errors.append("TEXT_OVERFLOW: text elements exceed safe zone boundaries")

    # 3. Logo clearspace
    if not check_logo_clearspace(design_flat, brief):
        errors.append("LOGO_CLEARSPACE: logo minimum margin violated")

    # 4. Mockup boundary — design stays within trophy surface
    if check_mockup_boundary(mockup, brief):
        warnings.append("MOCKUP_BOUNDARY: design slightly exceeds trophy surface")

    # 5. Output resolution
    W, H = mockup.size
    if min(W,H) < 1500:
        errors.append(f"RESOLUTION_LOW: {W}×{H}px. Minimum 1500px on short side.")

    return QAResult(passed=len(errors)==0, errors=errors, warnings=warnings)


def check_contrast_ratio(img: Image, brief: dict) -> float:
    """WCAG contrast ratio between text color and background."""
    def relative_luminance(hex_color):
        r,g,b = hex_to_rgb(hex_color)
        def f(c):
            c = c/255
            return c/12.92 if c <= 0.03928 else ((c+0.055)/1.055)**2.4
        return 0.2126*f(r) + 0.7152*f(g) + 0.0722*f(b)

    L1 = relative_luminance(brief["text_color"])
    L2 = relative_luminance(brief["background_color_1"])
    bright, dark = max(L1,L2), min(L1,L2)
    return (bright + 0.05) / (dark + 0.05)


async def qa_loop(briefs, designs, mockups, normalized_input, max_retries=3):
    """If QA fails, refeed error to Layer 1 for correction."""
    results = []
    for i, (brief, design, mockup) in enumerate(zip(briefs, designs, mockups)):
        for attempt in range(max_retries):
            qa = run_qa(design, mockup, brief)
            if qa.passed:
                results.append({"brief": brief, "mockup": mockup, "qa": qa})
```

```
                break
            else:
                # refeed error to Layer 1
                brief = await correct_brief(brief, qa.errors, normalized_input)
                design = render_design(brief, normalized_input["assets"], canvas_px)
                mockup = composite_mockup(design, ...)
        else:
            # max retries reached — flag for manual review
            results.append({"brief": brief, "mockup": mockup, "qa": qa, "flagged": True})
    return results
```

Prompt de corrección — Capa 4 → Capa 1

> The following design brief failed quality checks. Correct the specific issues and return a new valid design_brief JSON. Do not change the overall creative direction — only fix the reported errors.
>
> ORIGINAL BRIEF:
> {{ original_brief_json }}
>
> QA ERRORS:
> {% for error in errors %}
> - {{ error }}
> {% endfor %}
>
> BRAND ANALYSIS (for color reference):
> {{ brand_analysis_json }}
>
> Return ONLY the corrected design_brief JSON.

---

7. Desglose completo de tareas

ID	Tarea	Capa	Horas	Prioridad
T0.1	Extracción de colores del logo (ColorThief + SVG parser)	0	4–6h	MVP
T0.2	Análisis de geometría del logo (lxml + heurísticas)	0	6–8h	MVP
T0.3	Fetch + análisis de URL corporativa	0	4–6h	MVP
T0.4	Extracción de brand book PDF (PyMuPDF)	0	4–6h	MVP
T0.5	Ensamblado normalized_input + validación Pydantic	0	3–4h	MVP
T1.1	System prompt + user prompt	1	8–12h	MVP

ID	Tarea	Capa	Horas	Prioridad
	Llamada 1A (brand analysis)			
T1.2	System prompt + user prompt Llamada 1B (archetypos)	1	6–10h	MVP
T1.3	System prompt + user prompt Llamada 1C (design briefs)	1	8–12h	MVP
T1.4	Orquestación de las 3 llamadas + retry logic	1	4–6h	MVP
T1.5	Validación Pydantic del JSON de salida	1	3–5h	MVP
T2.1	render_background (solid + gradient + white)	2	4–6h	MVP
T2.2	render_text_blocks (jerarquía + fuentes)	2	6–8h	MVP
T2.3	render_logo (PNG/SVG + tratamientos + posición)	2	6–8h	MVP
T2.4	render_color_band	2	2–3h	MVP
T2.5	render_pattern_bg (sunburst + scatter + concentric)	2	6–8h	V1
T2.6	render_brand_shape (formas derivadas del logo)	2	5–7h	V1
T2.7	render_typography_as_graphic (texto rotado)	2	5–7h	V1
T2.8	render_split_layout (zonas cromáticas)	2	3–5h	V1
T2.9	render_decorative_lines	2	2–3h	V1
T2.10	render_illustration_1 (asset gráfico hero)	2	5–8h	V1
T3.1	Herramienta de calibración de homografías	3	6–10h	MVP
T3.2	Calibración manual de 3 modelos (MVP)	3	6–9h	MVP
T3.3	composite_mockup (warp + blend + soft edges)	3	8–12h	MVP
T3.4	Calibración del resto	3	18–30h	V1

ID	Tarea	Capa	Horas	Prioridad
	de modelos (15+)			
T4.1	check_contrast_ratio	4	2–4h	MVP
T4.2	check_text_overflow + check_logo_clearspace	4	4–6h	MVP
T4.3	check_mockup_boundary + resolution check	4	3–4h	V1
T4.4	qa_loop con refeed a Capa 1	4	4–6h	V1
—	Pipeline end-to-end + testing con brand books reales	—	16–24h	MVP
—	Ajuste de prompts post-testing (iteración)	—	12–20h	MVP

Fase	Tareas	Horas	Timeline
MVP	T0.1–T0.5, T1.1–T1.5, T2.1–T2.4, T3.1–T3.3, T4.1–T4.2 + pipeline + ajuste	74–105h	4–5 semanas
V1 completa	Todas las tareas anteriores + T2.5–T2.10, T3.4, T4.3–T4.4	160–230h	8–11 semanas total

✓ El MVP cubre los 3 recursos gráficos más frecuentes en el catálogo (solid bg, gradient bg, color_band) y los 2 patrones de texto más comunes. Suficiente para validar el sistema con clientes reales antes de invertir en los recursos más complejos.